

Terraform-04

1. Watch Terraform-04 video.

2. Execute the script shown in the video.ss

Version Constraints:

=====

Changing in terraform providers version may get us into incompatibility issues.

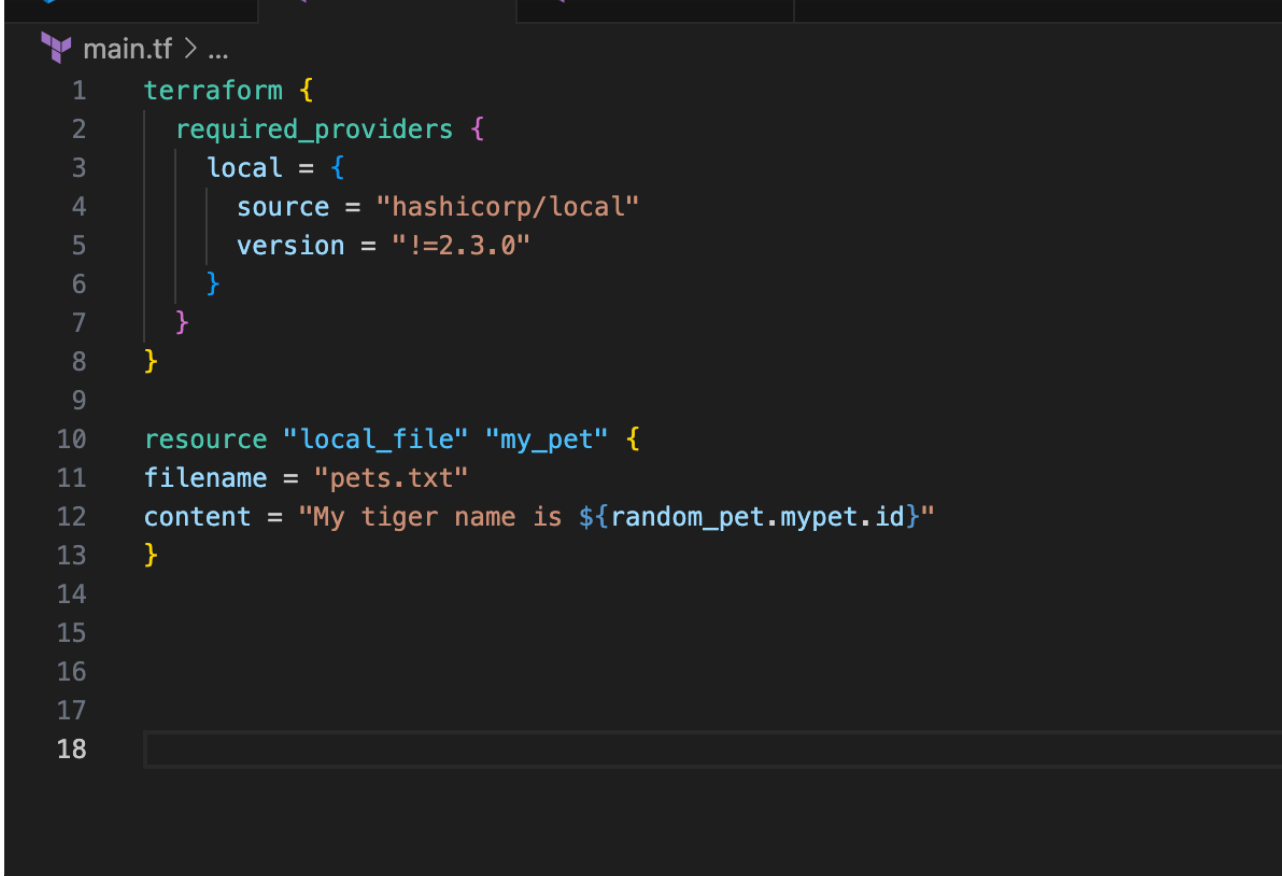
By default terraform will always try to download the latest version of provider available from registry.

To make sure to use the specific version provider we can add the provider block in configuration.

Example:

=====

```
terraform {  
  required_providers {  
    local = {  
      source = "hashicorp/local"  
      version = "2.3.0"  
    }  
  }  
}
```



```
main.tf > ...  
1  terraform {  
2      required_providers {  
3          local = {  
4              source = "hashicorp/local"  
5              version = "!=2.3.0"  
6          }  
7      }  
8  }  
9  
10 resource "local_file" "my_pet" {  
11     filename = "pets.txt"  
12     content = "My tiger name is ${random_pet.mypet.id}"  
13 }  
14  
15  
16  
17  
18
```

```

waseemakram@waseem terraform % terraform init
Initializing the backend...
Initializing provider plugins...
- Reusing previous version of hashicorp/local from the dependency lock file
- Reusing previous version of hashicorp/random from the dependency lock file
- Using previously-installed hashicorp/random v3.7.2

Error: Failed to query available provider packages

Could not retrieve the list of available versions for provider hashicorp/local: locked provider registry.terraform.io/hashicorp/local 2.6.1 does not match configured version 2.3.0; must use terraform init -upgrade to allow selection of new versions

To see which modules are currently depending on hashicorp/local and what versions are specified, run the following command:
    terraform providers

```

terraform providers

```

waseemakram@waseem terraform % terraform init -upgrade
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/local versions matching "2.3.0"...
- Finding latest version of hashicorp/random...
- Installing hashicorp/local v2.3.0...
- Installed hashicorp/local v2.3.0 (signed by HashiCorp)
- Installing hashicorp/random v3.8.0...
- Installed hashicorp/random v3.8.0 (signed by HashiCorp)
Terraform has made some changes to the provider dependency selections recorded
in the .terraform.lock.hcl file. Review those changes and commit them to your
version control system if they represent changes you intended to make.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other

```

version = "2.3.0" --> download the exact version

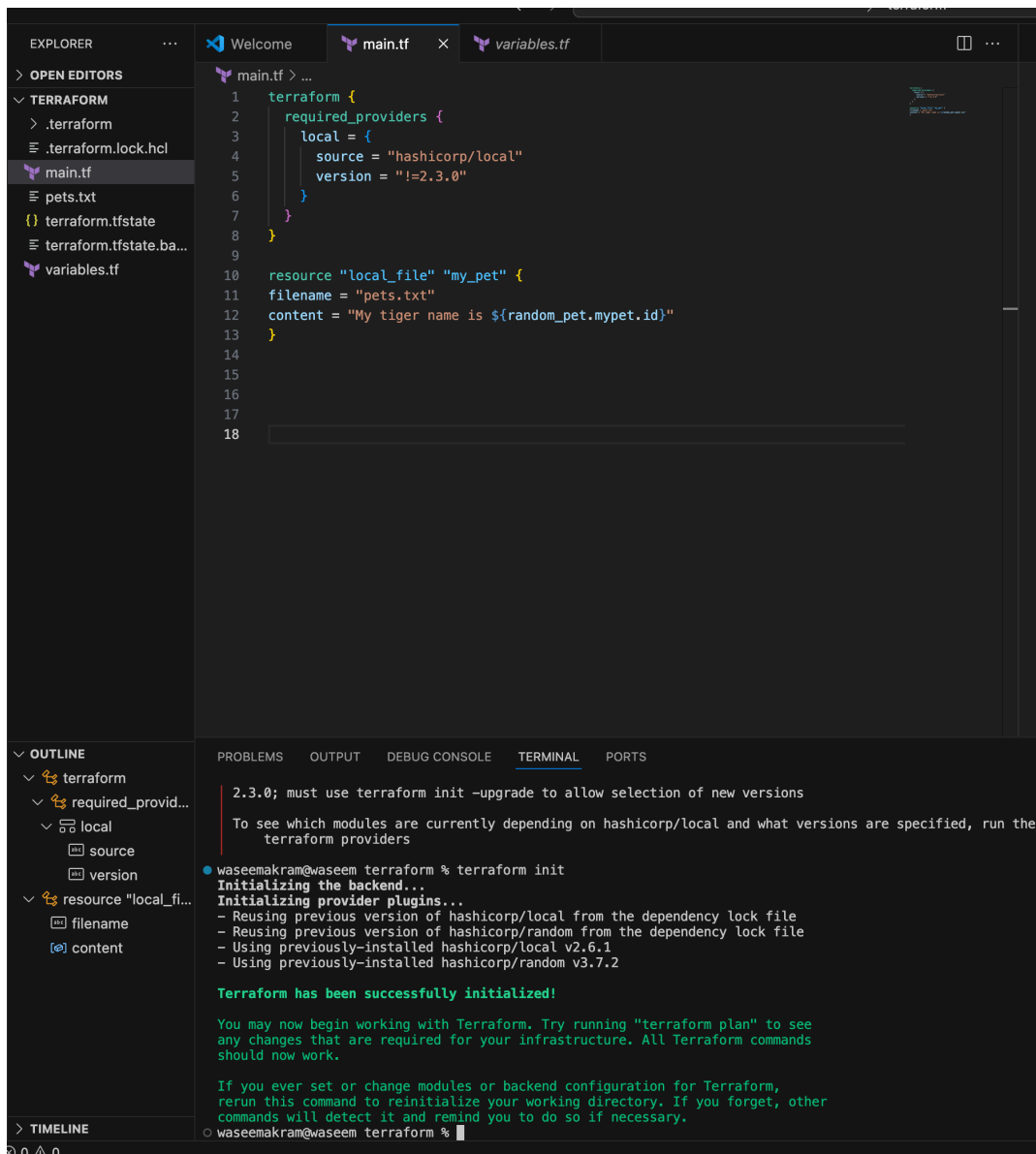
version = "!=2.3.0" --> will not use the mentioned version

version = "< 2.3.0" --> lesses than the mention version

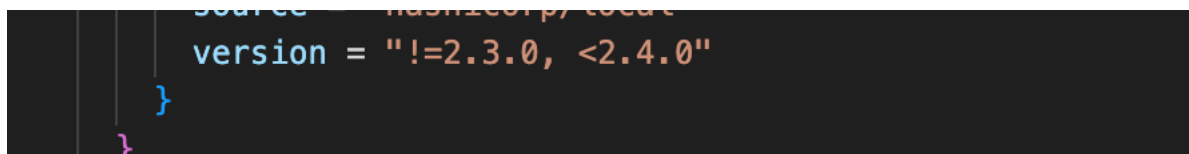
version = "> 2.3.0" --> greater than the given version

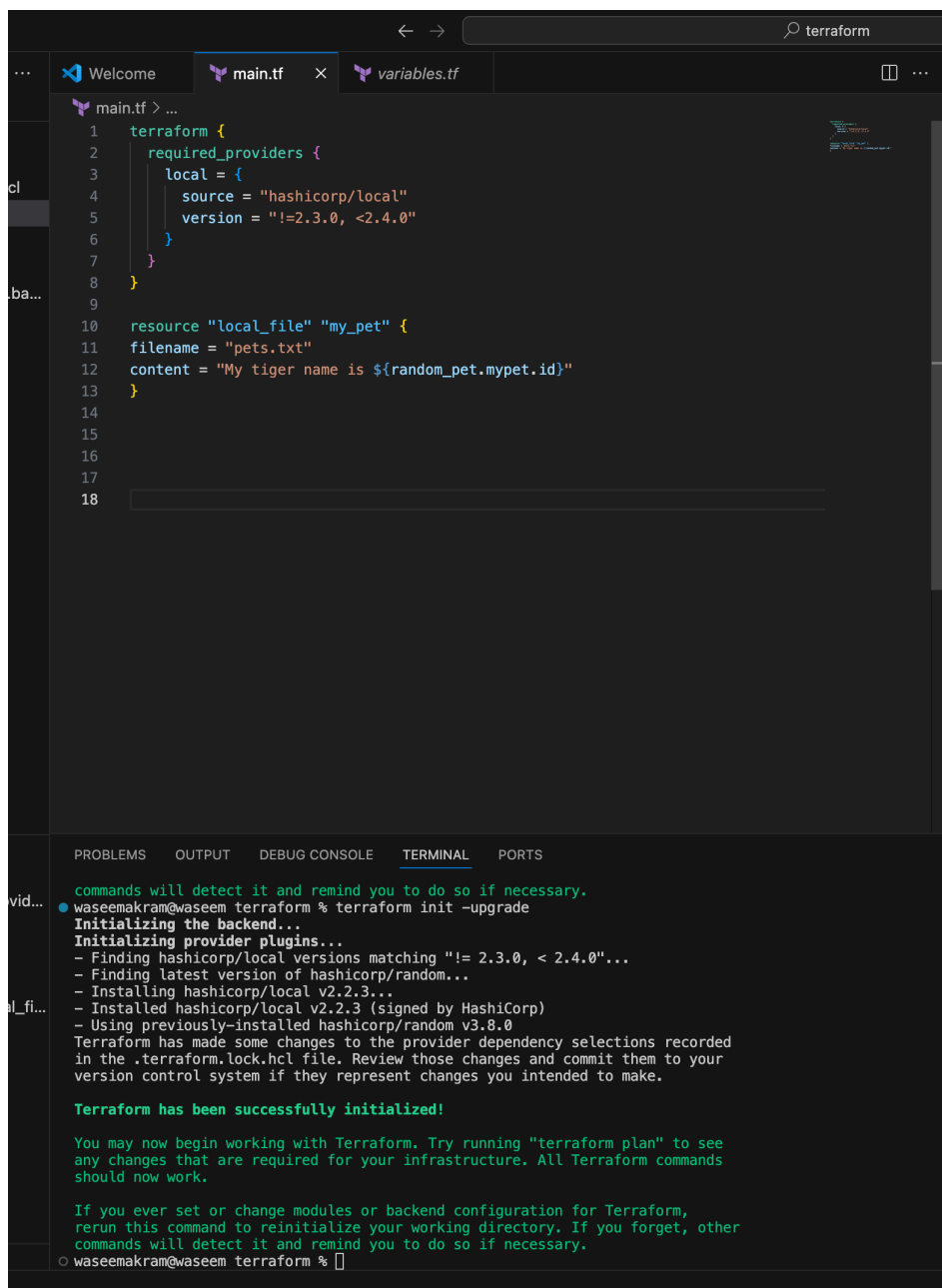
version = "~> 2.3.0" --> specific version or higher version.

—>



→





The screenshot shows a code editor with two tabs: 'main.tf' and 'variables.tf'. The 'main.tf' tab is active, displaying a Terraform configuration. The configuration includes a 'terraform' block with 'required_providers' and a 'resource' block for 'local_file' named 'my_pet'. The terminal at the bottom shows the output of the 'terraform init -upgrade' command, indicating that the providers have been successfully initialized.

```
1 terraform {
2   required_providers {
3     local = {
4       source = "hashicorp/local"
5       version = "!=2.3.0, <2.4.0"
6     }
7   }
8 }
9
10 resource "local_file" "my_pet" {
11   filename = "pets.txt"
12   content = "My tiger name is ${random_pet.mypet.id}"
13 }
14
15
16
17
18
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```
commands will detect it and remind you to do so if necessary.
● waseemakram@waseem terraform % terraform init -upgrade
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/local versions matching "!= 2.3.0, < 2.4.0"...
- Finding latest version of hashicorp/random...
- Installing hashicorp/local v2.2.3...
- Installed hashicorp/local v2.2.3 (signed by HashiCorp)
- Using previously-installed hashicorp/random v3.8.0
Terraform has made some changes to the provider dependency selections recorded
in the .terraform.lock.hcl file. Review those changes and commit them to your
version control system if they represent changes you intended to make.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
○ waseemakram@waseem terraform %
```

Data sources:

=====

Apart from terraform we have multiple other tools where the infra can be created.

Ex: ansible,salt,puppet,bash script>manual process.

Data sources are used to read the content of the infrastructure

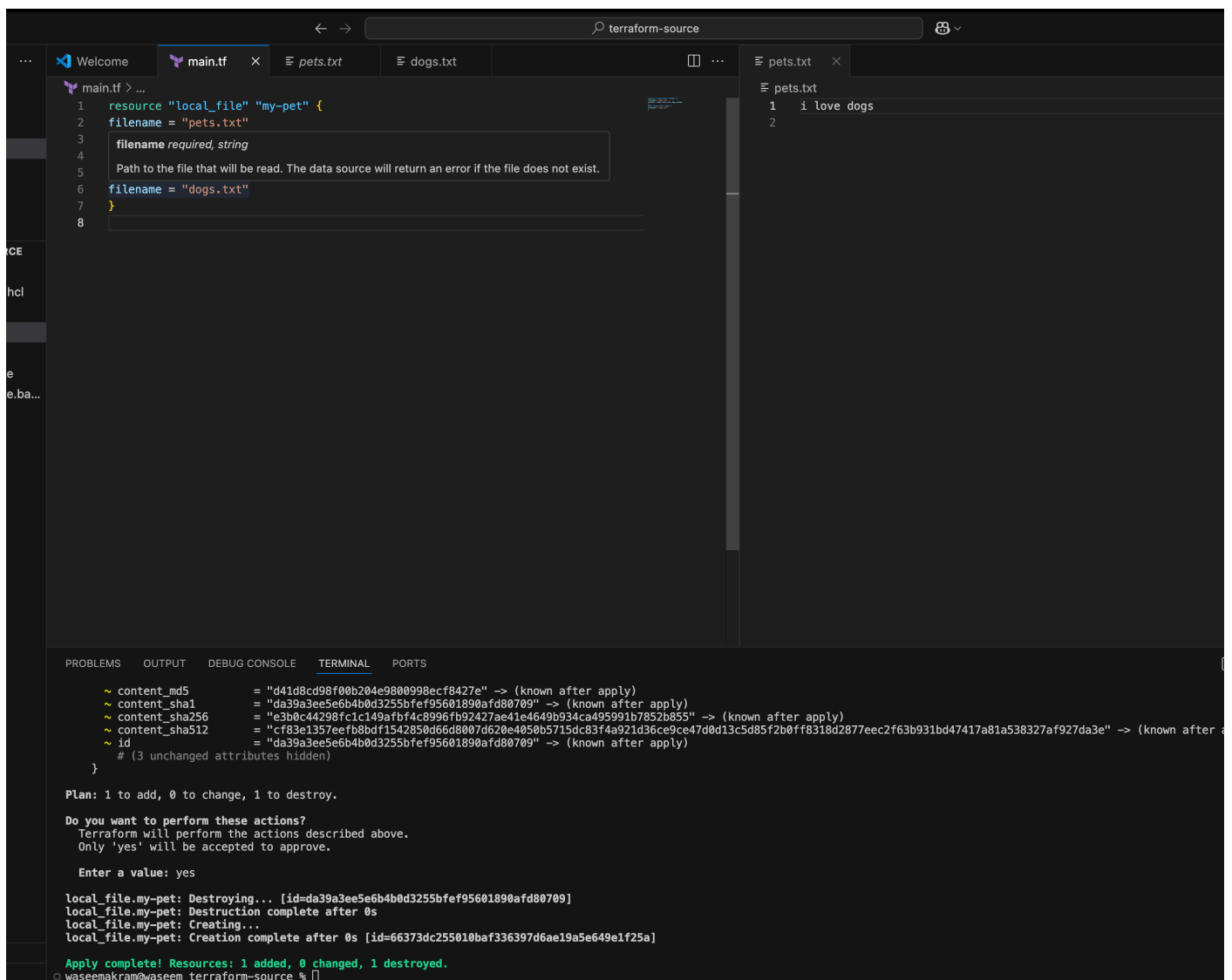
for example if we want terraform to read the content of the file which has been created by any other tool.

create a file called in dogs.txt in the same terraform working directory.

main.tf

=====

```
resource "local_file" "my-pet" {
  filename = "pets.txt"
  content = data.local_file.dog.content
}
data "local_file" "dog" {
  filename = "dogs.txt"
}
```



The screenshot shows a VS Code editor with a dark theme. The top bar shows the file explorer with tabs for 'main.tf', 'pets.txt', and 'dogs.txt'. The 'main.tf' tab is active, displaying the Terraform configuration code. A tooltip is visible over the 'filename' attribute of the 'local_file' resource, stating 'filename required, string' and 'Path to the file that will be read. The data source will return an error if the file does not exist.' The 'pets.txt' tab is also open, showing the content '1 i love dogs' and '2'. The bottom panel shows the 'TERMINAL' output, which includes the Terraform plan and apply results. The plan shows that the 'local_file.my-pet' resource is being destroyed and recreated. The apply output shows the resource being created successfully.

```
main.tf > ...
1 resource "local_file" "my-pet" {
2   filename = "pets.txt"
3
4   filename required, string
5   Path to the file that will be read. The data source will return an error if the file does not exist.
6   filename = "dogs.txt"
7 }
8

~ content_md5      = "d41d8cd98f00b204e9800998ecf8427e" -> (known after apply)
~ content_sha1     = "da39a3ee5e6b4b0d3255bfef95601890afd80709" -> (known after apply)
~ content_sha256   = "e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855" -> (known after apply)
~ content_sha512   = "cf83e1357eefb8bd71542850d66d8007d620e4050b5715dc83f4a921d36ce9ce47d0d13c5d85f2b0ff8318d2877e2cf63b931bd47417a81a538327af927da3e" -> (known after apply)
~ id              = "da39a3ee5e6b4b0d3255bfef95601890afd80709" -> (known after apply)
# (3 unchanged attributes hidden)
}

Plan: 1 to add, 0 to change, 1 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

Enter a value: yes

local_file.my-pet: Destroying... [id=da39a3ee5e6b4b0d3255bfef95601890afd80709]
local_file.my-pet: Destruction complete after 0s
local_file.my-pet: Creating...
local_file.my-pet: Creation complete after 0s [id=66373dc255010baf336397d6ae19a5e649e1f25a]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
waseemakram@waseem terraform-source %
```

Meta-Arguments:

=====

Meta arguments are used if we want to create multiple resources.

Meta arguments can be used within any resource block to change the behaviour of the resources.

Examples for meta arguments:

- 1) Depends_on
- 2) lifecycle rules
- 3) Count
- 4) For_each

Example of count:

=====

If we use count as 3 then it will create 3 files with pet[0],pet[1],pet[2]

The screenshot shows a VS Code editor with a Terraform project. The Explorer sidebar on the left shows the file structure: .terraform, .terraform.lock.hcl, cats.txt, dogs.txt, main.tf, terraform.tfstate, terraform.tfstate.backup, variables.tf, and whitetiger.txt. The main editor displays the main.tf file with the following content:

```
1 resource "local_file" "my-pet" {
2   filename = var.filename[count.index]
3   content = "i love whitetigers!"
4   count = length(var.filename)
5 }
6
7
8
9
10
```

The whitetiger.txt file is open in the editor, showing the content: "1 i love whitetigers!". The terminal at the bottom shows the Terraform execution output:

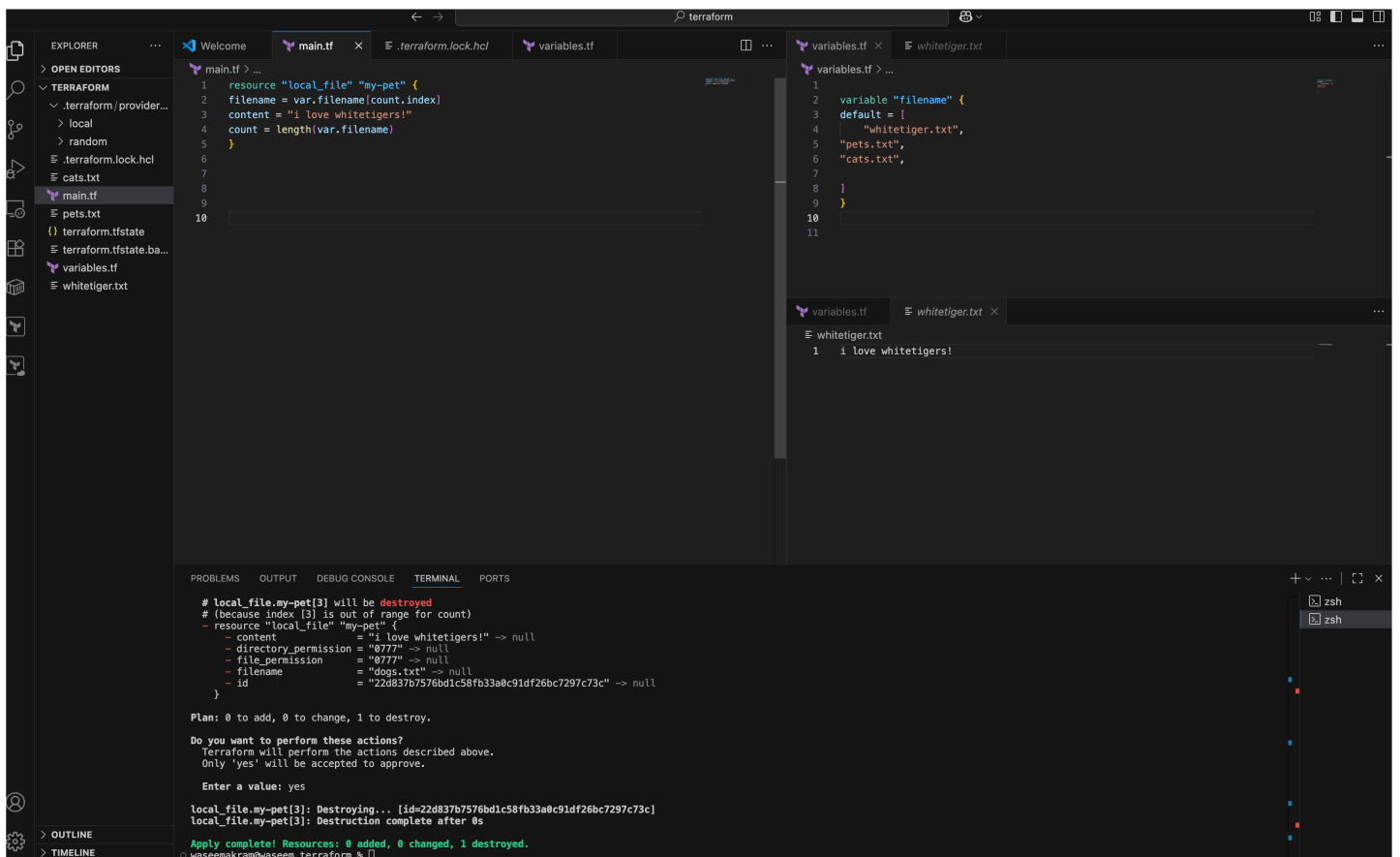
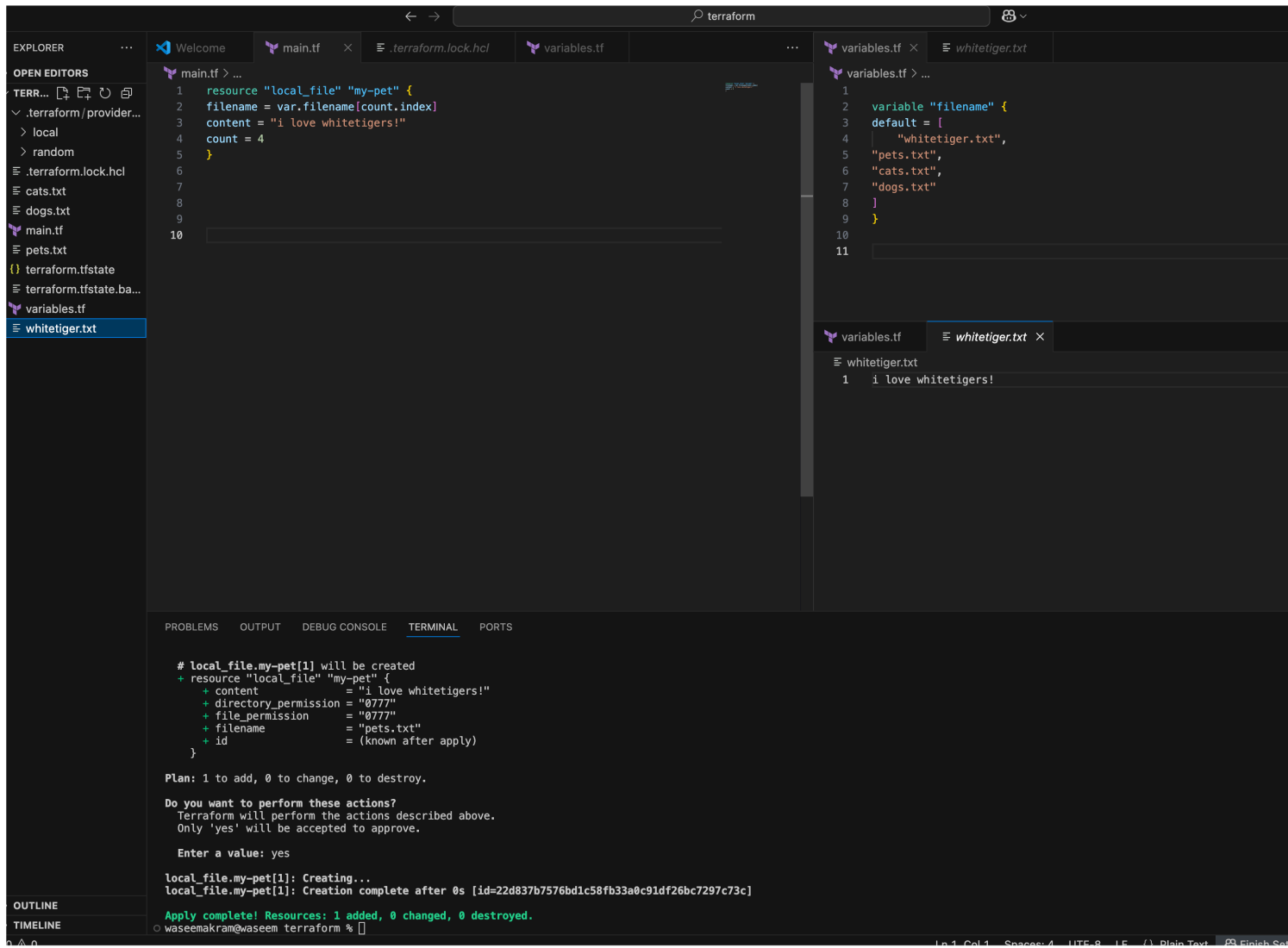
```
- pet_name = "MR.snipe" -> null

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

local_file.my_pet: Destroying... [id=aae55b98fd53968b11bd8b26b00fba48255a5d5a]
local_file.my-pet[0]: Creating...
local_file.my-pet[2]: Creating...
local_file.my-pet[1]: Creating...
local_file.my-pet[3]: Creating...
local_file.my_pet: Destruction complete after 0s
local_file.my-pet[0]: Creation complete after 0s [id=22d837b7576bd1c58fb33a0c91df26bc7297c73c]
local_file.my-pet[1]: Creation complete after 0s [id=22d837b7576bd1c58fb33a0c91df26bc7297c73c]
random_pet.mypet: Destroying... [id=MR.snipe]
local_file.my-pet[3]: Creation complete after 0s [id=22d837b7576bd1c58fb33a0c91df26bc7297c73c]
local_file.my-pet[2]: Creation complete after 0s [id=22d837b7576bd1c58fb33a0c91df26bc7297c73c]
random_pet.mypet: Destruction complete after 0s

Apply complete! Resources: 4 added, 0 changed, 2 destroyed.
waseemakram@waseem terraform %
```



Example of for_each:

=====

The screenshot shows a VS Code editor with the following files open: `main.tf`, `pets.txt`, `variables.tf`, and `whitetiger.txt`. The `main.tf` file contains a Terraform configuration for creating local files. The `variables.tf` file defines a variable `filename` with a default value of `["whitetiger.txt", "pets.txt", "cats.txt"]`. The `whitetiger.txt` file contains the text `i love whitetigers and wolf`. The terminal output shows the execution of the Terraform plan and apply commands, resulting in the creation of three local files: `local_file.my-pet[0]`, `local_file.my-pet[1]`, and `local_file.my-pet[2]`. The output also shows the destruction of three local files: `local_file.my-pet[0]`, `local_file.my-pet[1]`, and `local_file.my-pet[2]`.

```
main.tf > ...
1 resource "local_file" "pet" {
2   filename = each.value
3   content = "i love whitetigers and wolf"
4
5   for_each = toset(var.filename)
6 }
7
8
9
10
11
12

variables.tf > ...
1
2 variable "filename" {
3   type = set(string)
4   default = [
5     "whitetiger.txt",
6     "pets.txt",
7     "cats.txt",
8   ]
9 }
10
11
12

variables.tf > ...
whitetiger.txt
1 i love whitetigers and wolf

Plan: 3 to add, 0 to change, 3 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

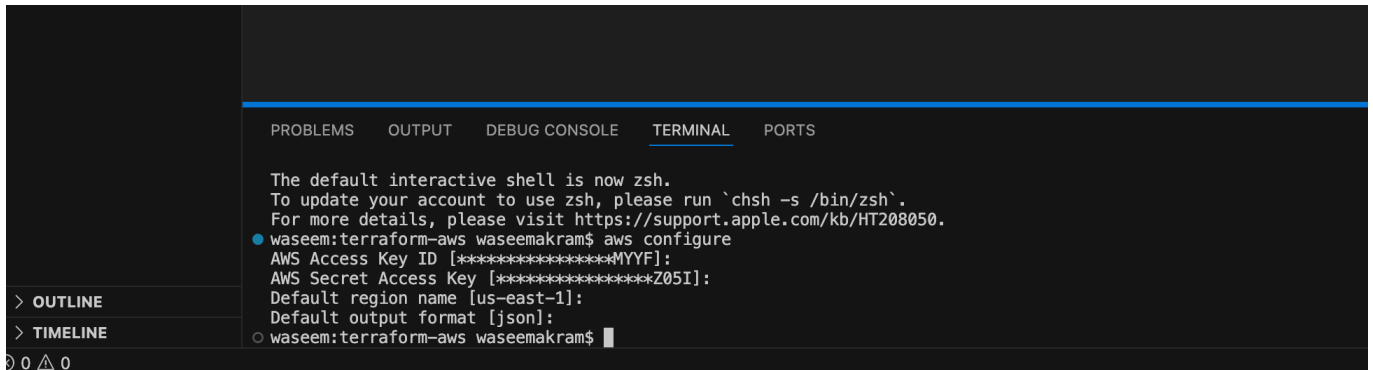
Enter a value: yes
local_file.my-pet[1]: Destroying... [id=22d837b7576bdc58fb33a0c91df26bc7297c73c]
local_file.my-pet[0]: Destroying... [id=22d837b7576bdc58fb33a0c91df26bc7297c73c]
local_file.my-pet[2]: Destroying... [id=22d837b7576bdc58fb33a0c91df26bc7297c73c]
local_file.pet["cats.txt"]: Creating...
local_file.pet["whitetiger.txt"]: Creating...
local_file.pet["pets.txt"]: Creating...
local_file.my-pet[0]: Destruction complete after 0s
local_file.my-pet[1]: Destruction complete after 0s
local_file.pet["whitetiger.txt"]: Creation complete after 0s [id=f98512fa47eea34645c553e49751dbcdc4d43542]
local_file.my-pet[2]: Destruction complete after 0s
local_file.pet["pets.txt"]: Creation complete after 0s [id=f98512fa47eea34645c553e49751dbcdc4d43542]
local_file.pet["cats.txt"]: Creation complete after 0s [id=f98512fa47eea34645c553e49751dbcdc4d43542]

Apply complete! Resources: 3 added, 0 changed, 3 destroyed.
```


Terraform with AWS:

=====

--> first we need to create a secret key and access key and configure then in laptop.



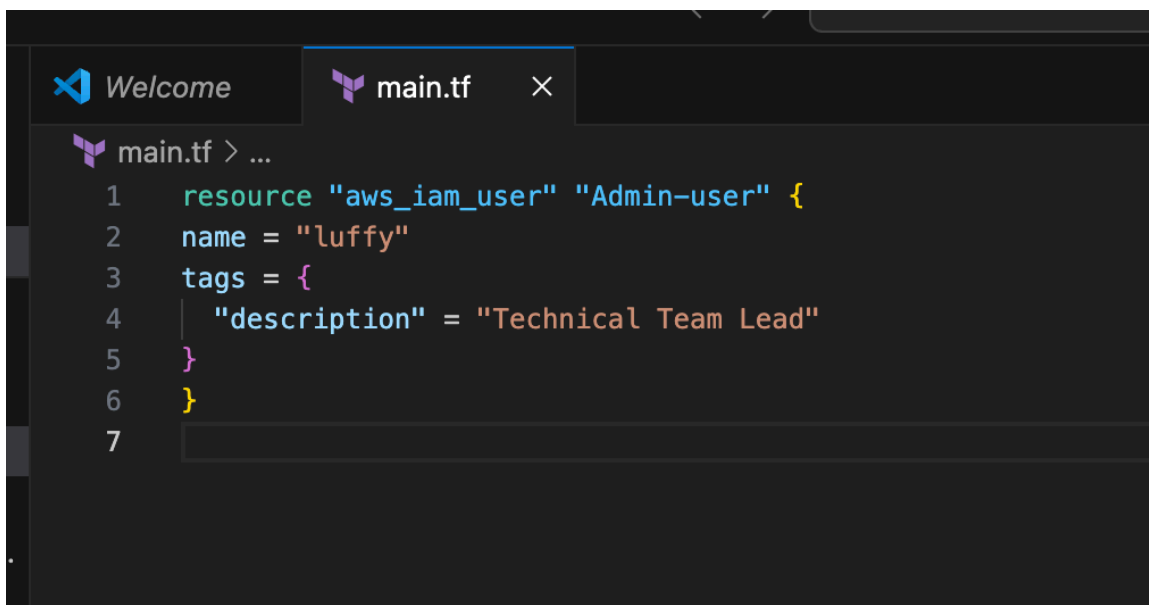
```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

The default interactive shell is now zsh.
To update your account to use zsh, please run `chsh -s /bin/zsh`.
For more details, please visit https://support.apple.com/kb/HT208050.
● waseem:terraform-aws waseemakram$ aws configure
AWS Access Key ID [*****MYF]:
AWS Secret Access Key [*****Z05I]:
Default region name [us-east-1]:
Default output format [json]:
○ waseem:terraform-aws waseemakram$
```

Example script to create aws iam:

=====

```
resource "aws_iam_user" "Admin-user" {
  name = "lucy"
  tags = {
    "description" = "Technical Team Lead"
  }
}
```



```
Welcome main.tf x
main.tf > ...
1 resource "aws_iam_user" "Admin-user" {
2   name = "luffy"
3   tags = {
4     "description" = "Technical Team Lead"
5   }
6 }
7
```

The image shows a VS Code editor window with a dark theme. The left sidebar displays the Explorer view with a file tree containing: `WELCOME`, `main.tf`, `terraform-aws`, `terraform.lock.hcl`, `main.tf`, `terraform.tfstate`, and `terraform.tfstate.backup`. The main editor area has two tabs: `Welcome` and `main.tf`. The `main.tf` tab is active, showing a Terraform configuration for an AWS IAM user:

```
1 resource "aws_iam_user" "Admin-user" {
2   name = "luffy"
3   tags = {
4     "description" = "Technical Team Lead"
5   }
6 }
7
```

Below the editor, the TERMINAL view is open, displaying the output of a Terraform command. The output includes a note about the `-out` option, a confirmation of the execution plan, and the actions Terraform will perform. The plan shows that the `aws_iam_user.Admin-user` resource will be created with the following attributes:

- `arn`: (known after apply)
- `force_destroy`: false
- `id`: (known after apply)
- `name`: "luffy"
- `path`: "/"
- `tags`: {"description": "Technical Team Lead"}
- `tags_all`: {"description": "Technical Team Lead"}
- `unique_id`: (known after apply)

The terminal output concludes with the plan summary: `Plan: 1 to add, 0 to change, 0 to destroy.` and a prompt to perform the actions. The user enters `yes`, and the terminal shows the successful creation of the `aws_iam_user.Admin-user` resource.

```
--> now check your aws account and inside--> iam check --> users
--> we will able to see puffy named user in iam
```

Identity and Access Management (IAM)

Users (5) Info

An IAM user is an identity with long-term credentials that is used to interact with AWS in an account.

Search IAM

Dashboard

Access Management

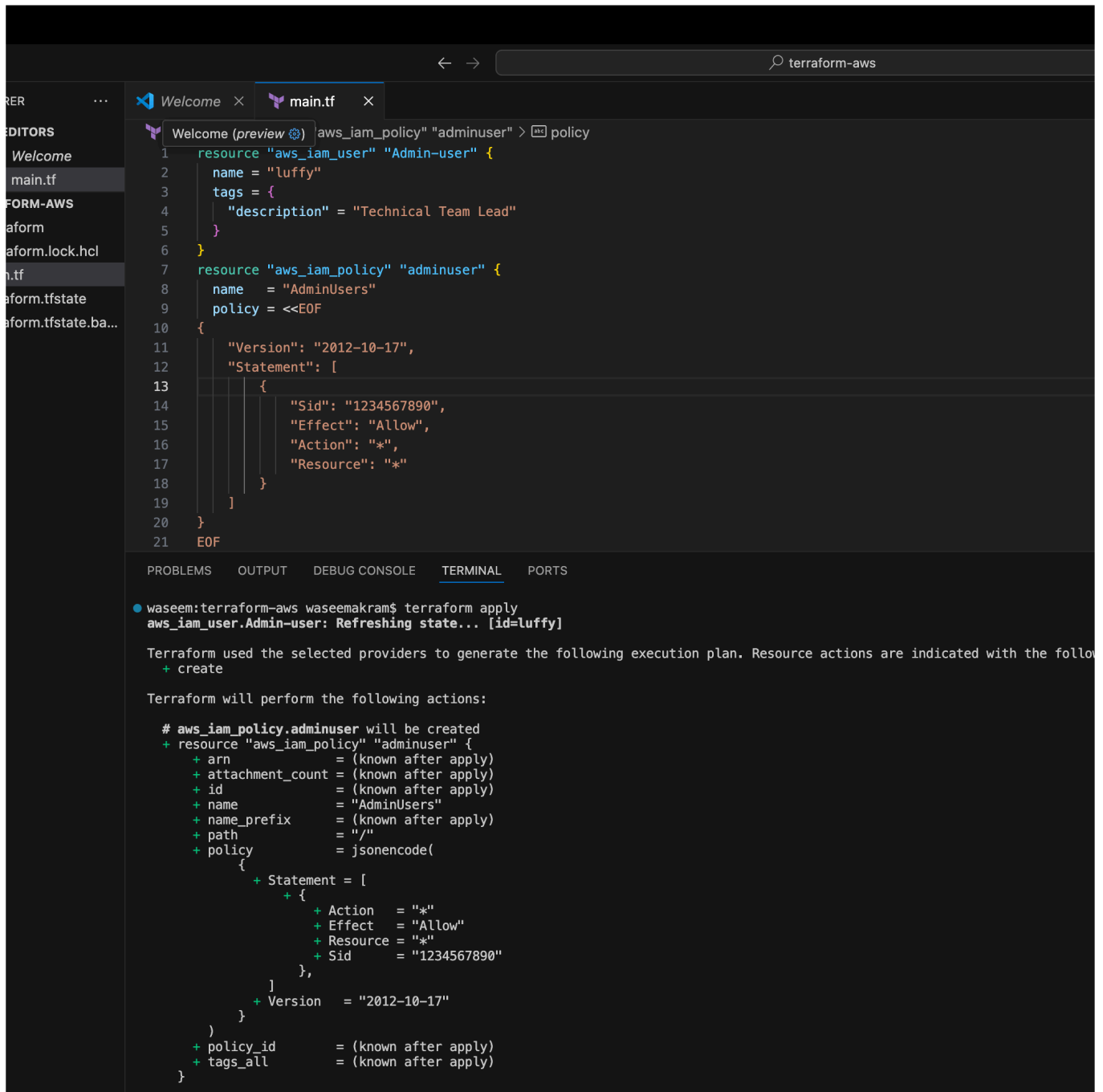
User groups

☐	User name ▲	Path ▼	Group: ▼	Last activity ▼	MFA ▼	Password age ▼	Console last sign-in ▼	Access key ID ▼
☐	devops	/	1	✓ 44 days ago	-	✓ 52 days	✓ 44 days ago	-
☐	luffy	/	0	-	-	-	-	-

Policy attach to the user

```
resource "aws_iam_user" "Admin-user" {
  name = "lucy"
  tags = {
    "description" = "Technical Team Lead"
  }
}
resource "aws_iam_policy" "adminuser" {
  name = "AdminUsers"
  policy = <<EOF
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "1234567890",
      "Effect": "Allow",
      "Action": "*",
      "Resource": "*"
    }
  ]
}
EOF
}

resource "aws_iam_user_policy_attachment" "lucy-admin-access"
{
  user      = aws_iam_user.Admin-user.name
  policy_arn = aws_iam_policy.adminuser.arn
}
```



```
# aws_iam_user_policy_attachment.luffy-admin-access will be created
+ resource "aws_iam_user_policy_attachment" "luffy-admin-access" {
+   id           = (known after apply)
+   policy_arn   = (known after apply)
+   user         = "luffy"
+ }

Plan: 2 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_iam_policy.adminuser: Creating...
aws_iam_policy.adminuser: Creation complete after 2s [id=arn:aws:iam::207662791773:policy/AdminUsers]
aws_iam_user_policy_attachment.luffy-admin-access: Creating...
aws_iam_user_policy_attachment.luffy-admin-access: Creation complete after 1s [id=luffy-20260119092823547600000001]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
○ waseem:terraform-aws waseemakram$
```

--> now check the aws account and check the policy which we added to user named luffy

Users

luffy

Info

Delete

Summary

ARN

arn:aws:iam::207662791773:user/luffy

Console access

Disabled

Access key 1

Create access key

Created

January 19, 2026, 14:49 (UTC+05:30)

Last console sign-in

-

Permissions

Groups

Tags (1)

Security credentials

Last Accessed

Permissions policies (1)

Remove

Add permissions

Permissions are defined by policies attached to the user directly or through groups.

Filter by Type

All types

Search

Policy name

Type

Attached via

AdminUsers

Customer managed

Directly

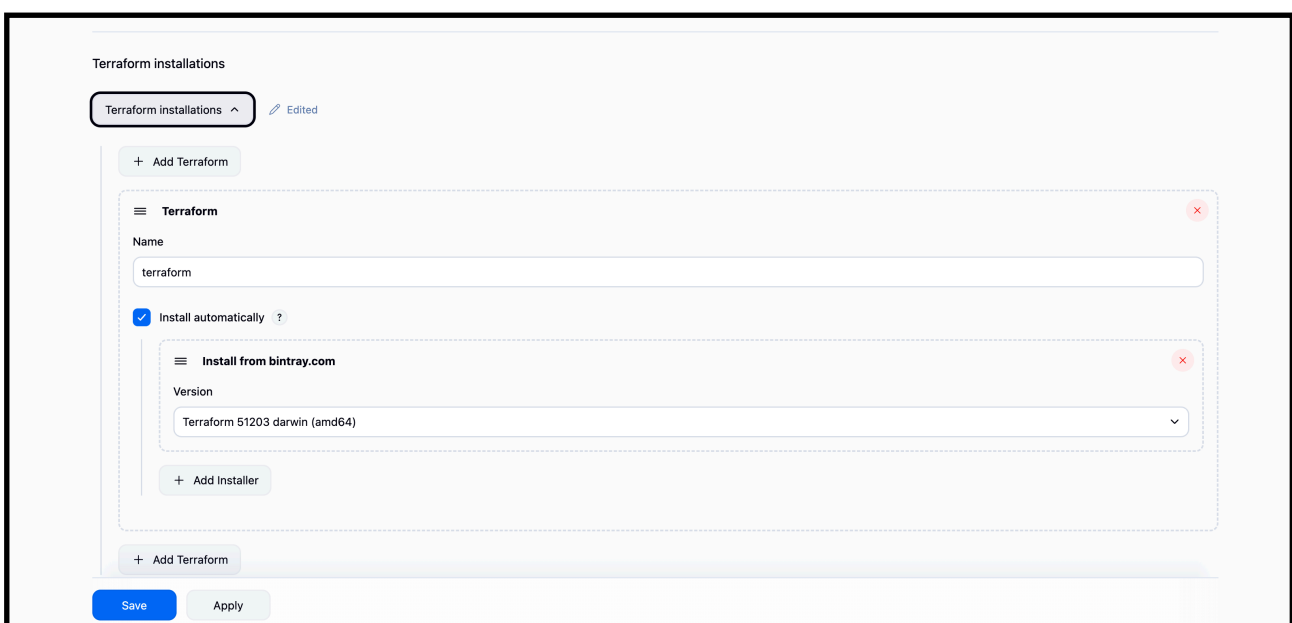
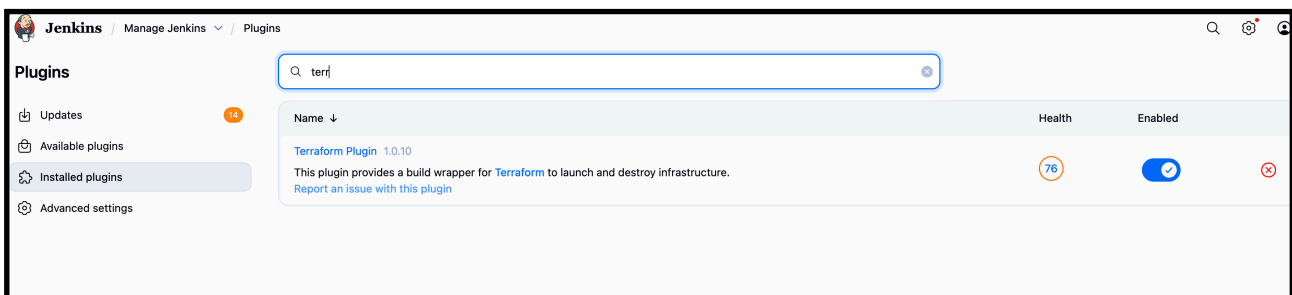
Permissions boundary (not set)

3.Integrate Terraform in Jenkins using the Terraform plugin.

Step 1: Install the Terraform Plugin in Jenkins

1. Go to **Jenkins Dashboard** → **Manage Jenkins** → **Plugins** → **Available Plugins**.
2. Search for **Terraform**.
3. Install **Terraform Plugin**
4. Restart Jenkins if required.

—-> we are creating a user named zero in our aws account with this task using Jenkins pipeline

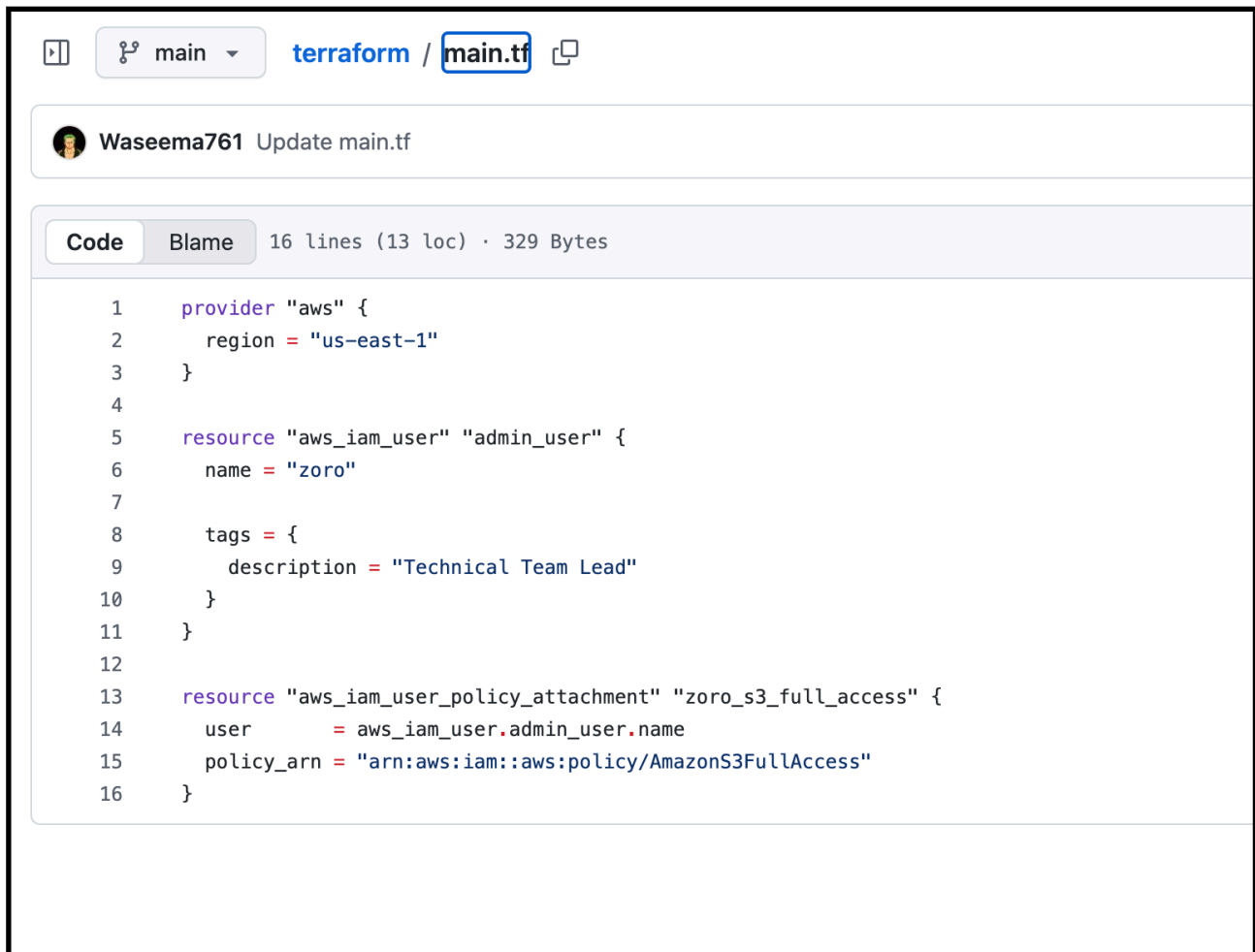


Step 2: Create a Jenkins Job (Pipeline or Freestyle)

You can use either **Freestyle** or **Pipeline**.

—> inside GitHub

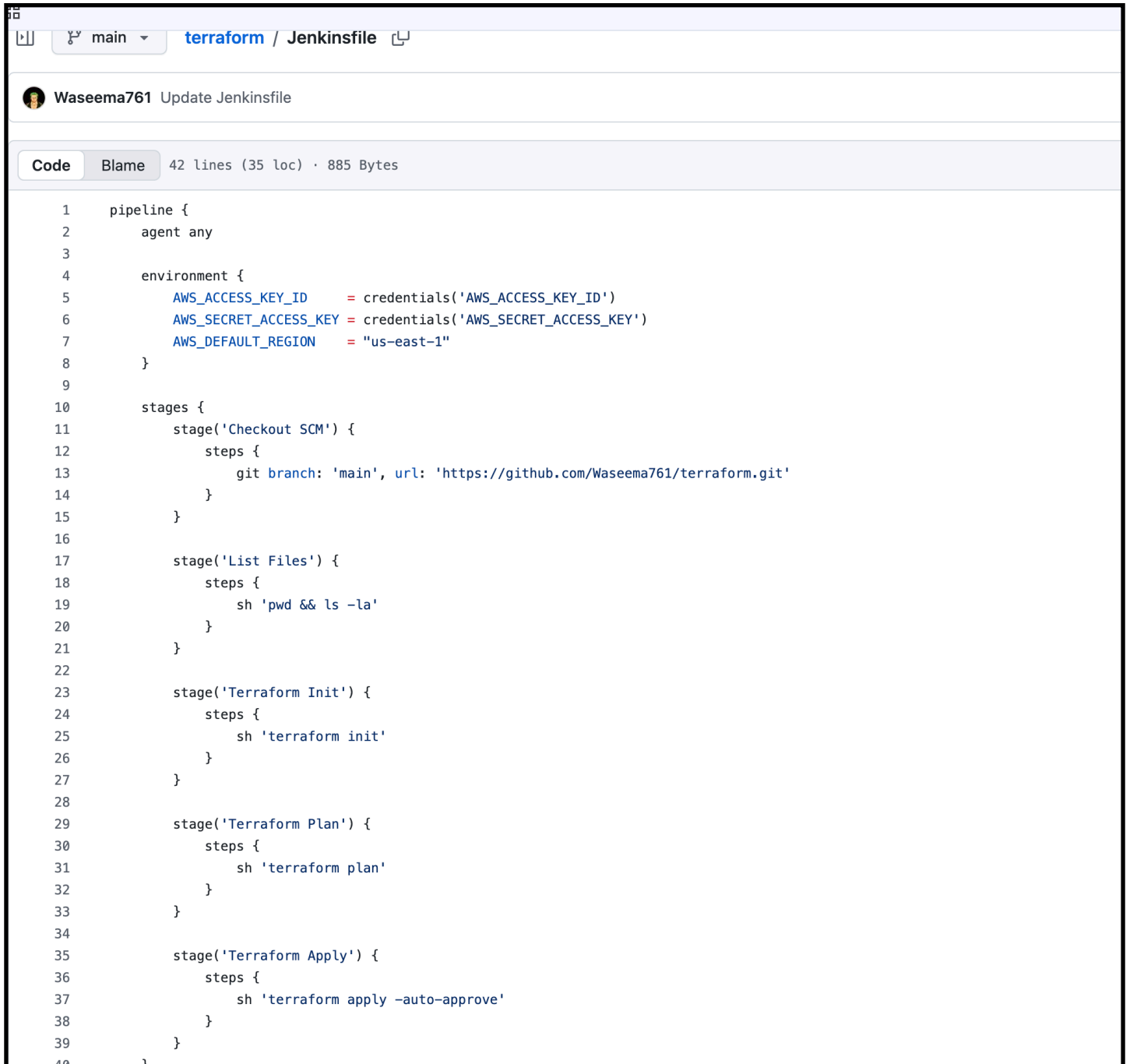
—> add this code in main.tf in your repo file



The screenshot shows a GitHub repository interface. At the top, there's a navigation bar with a repository icon, a dropdown menu set to 'main', and the path 'terraform / main.tf'. Below this, a commit message 'Waseema761 Update main.tf' is visible. The main content area shows the 'Code' tab selected, displaying the contents of 'main.tf'. The file is 16 lines long (13 loc) and 329 bytes. The code is Terraform configuration for AWS IAM user and policy attachment.

```
1  provider "aws" {
2      region = "us-east-1"
3  }
4
5  resource "aws_iam_user" "admin_user" {
6      name = "zoro"
7
8      tags = {
9          description = "Technical Team Lead"
10     }
11 }
12
13 resource "aws_iam_user_policy_attachment" "zoro_s3_full_access" {
14     user          = aws_iam_user.admin_user.name
15     policy_arn    = "arn:aws:iam::aws:policy/AmazonS3FullAccess"
16 }
```

jenkinsfile



The screenshot shows a GitHub repository page for the 'terraform / Jenkinsfile' file. The repository is owned by 'Waseema761'. The file is named 'Jenkinsfile' and is located in the 'main' branch. The file has 42 lines (35 loc) and 885 Bytes. The file content is a Jenkinsfile script for a Terraform pipeline. The script defines a pipeline with an agent 'any' and an environment with AWS credentials and region. The pipeline consists of several stages: 'Checkout SCM', 'List Files', 'Terraform Init', 'Terraform Plan', and 'Terraform Apply'. Each stage contains specific steps for executing the Terraform commands.

```
1 pipeline {
2   agent any
3
4   environment {
5     AWS_ACCESS_KEY_ID = credentials('AWS_ACCESS_KEY_ID')
6     AWS_SECRET_ACCESS_KEY = credentials('AWS_SECRET_ACCESS_KEY')
7     AWS_DEFAULT_REGION = "us-east-1"
8   }
9
10  stages {
11    stage('Checkout SCM') {
12      steps {
13        git branch: 'main', url: 'https://github.com/Waseema761/terraform.git'
14      }
15    }
16
17    stage('List Files') {
18      steps {
19        sh 'pwd && ls -la'
20      }
21    }
22
23    stage('Terraform Init') {
24      steps {
25        sh 'terraform init'
26      }
27    }
28
29    stage('Terraform Plan') {
30      steps {
31        sh 'terraform plan'
32      }
33    }
34
35    stage('Terraform Apply') {
36      steps {
37        sh 'terraform apply -auto-approve'
38      }
39    }
40  }
```


—> make sure you created credentials of your aws access key details in Jenkins gui credentials

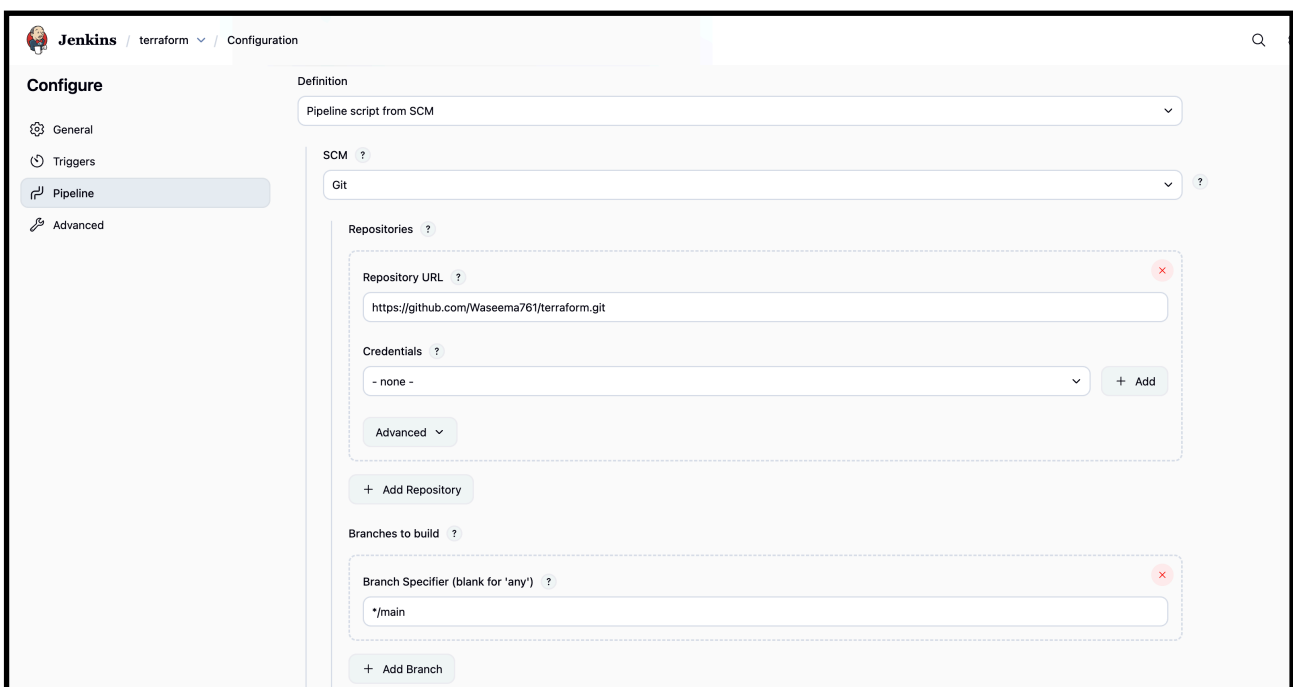
	AWS_ACCESS_KEY_ID	AWS_ACCESS_KEY_ID	Secret text	
	AWS_SECRET_ACCESS_KEY	AWS_SECRET_ACCESS_KEY	Secret text	

—> access key id : your access key id attach it secret text

—> secret access key : your aws access secret key attach it with secret text credential

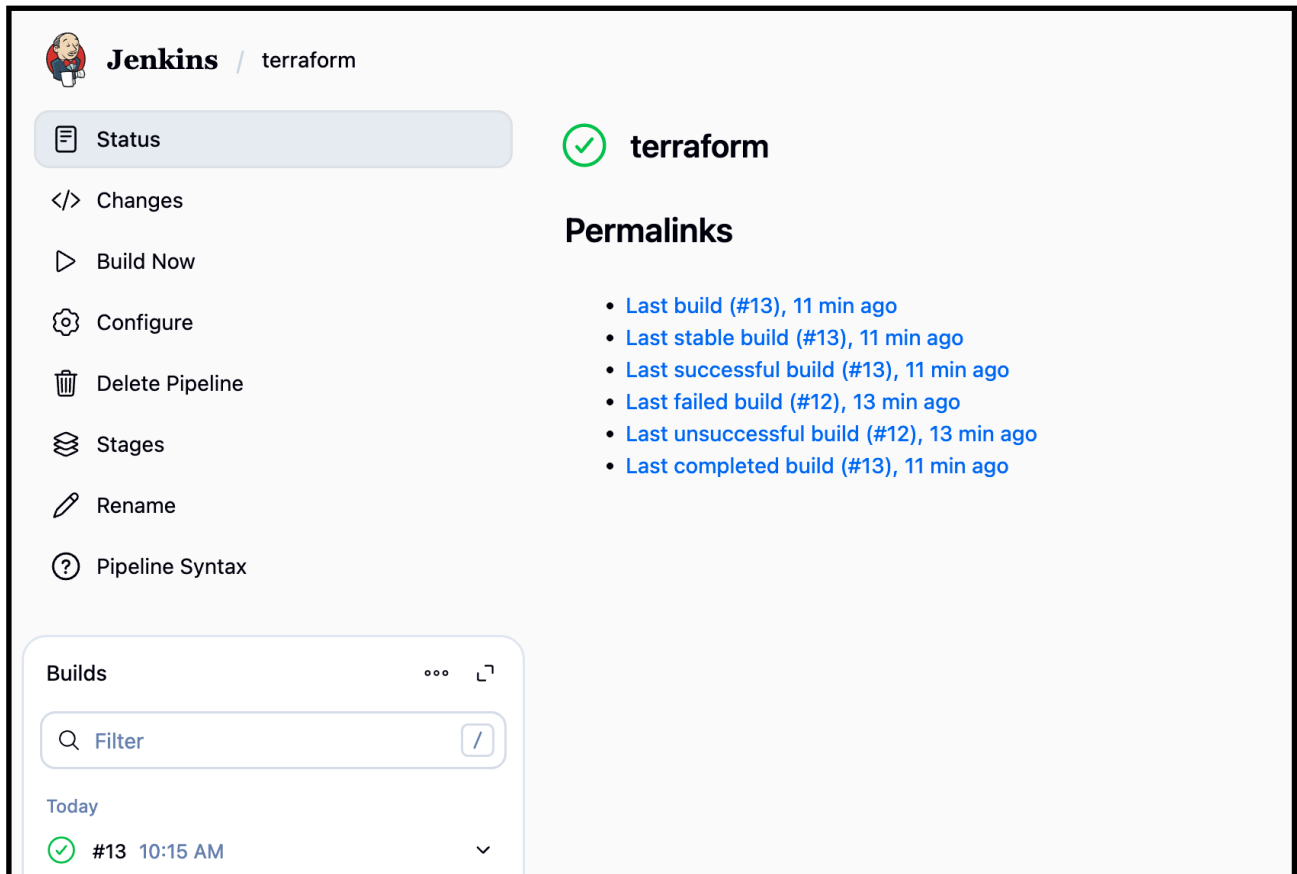
Now create a job and build :

—> attach your repo were we added jenkinsfile and main.tf



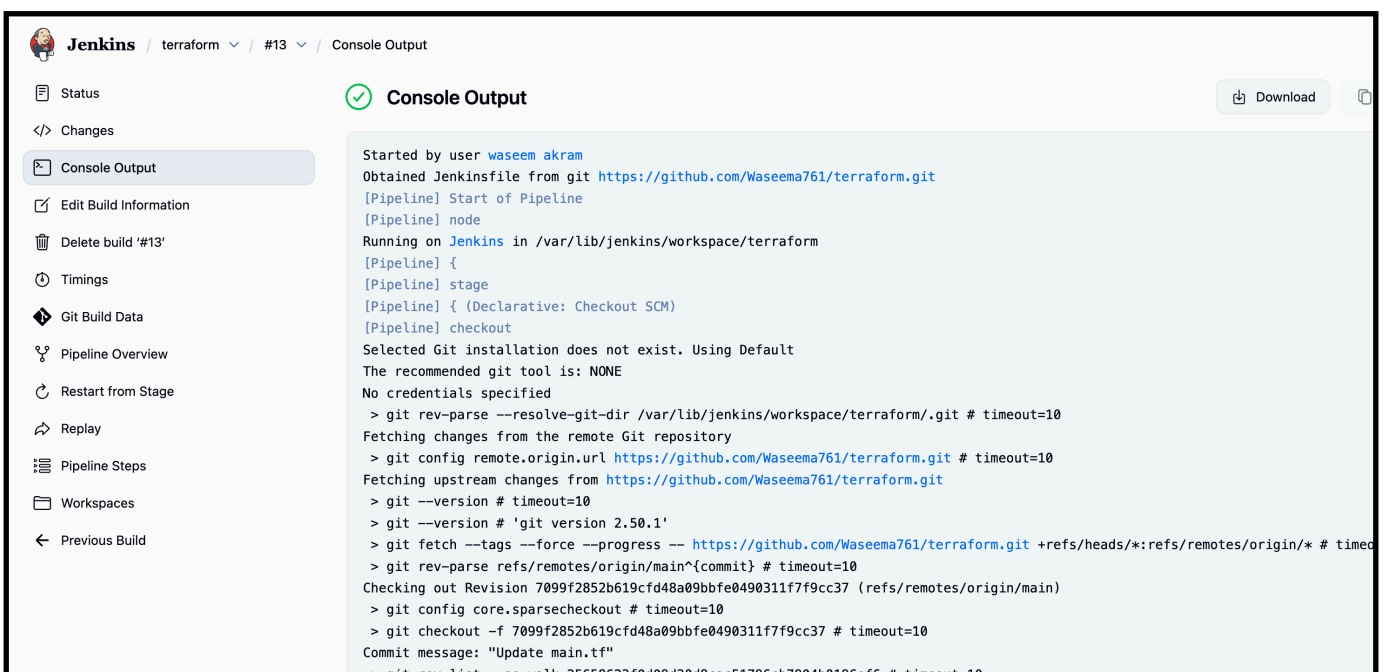
The image shows the Jenkins Configuration page for a Pipeline job named 'terraform'. The left sidebar contains the 'Configure' section with tabs for General, Triggers, Pipeline, and Advanced. The 'Pipeline' tab is selected. The main area shows the 'Definition' section with a dropdown menu set to 'Pipeline script from SCM'. Below this, the 'SCM' section is set to 'Git'. The 'Repositories' section contains a single repository with the URL 'https://github.com/Waseema761/terraform.git' and the 'Credentials' dropdown set to '- none -'. There is an 'Add Repository' button below. The 'Branches to build' section has a 'Branch Specifier (blank for \'any\')' dropdown set to '*/main' and an 'Add Branch' button below.

—> build job :



The screenshot shows the Jenkins web interface for a job named 'terraform'. On the left, a sidebar contains navigation links: Status (selected), Changes, Build Now, Configure, Delete Pipeline, Stages, Rename, and Pipeline Syntax. The main area features a green checkmark icon and the job name 'terraform'. Below this, a 'Permalinks' section lists several links for the latest build (#13), including 'Last build (#13), 11 min ago', 'Last stable build (#13), 11 min ago', 'Last successful build (#13), 11 min ago', 'Last failed build (#12), 13 min ago', 'Last unsuccessful build (#12), 13 min ago', and 'Last completed build (#13), 11 min ago'. At the bottom, a 'Builds' section shows a table with one entry: build #13, completed at 10:15 AM, with a green status icon.

—> check the console output:



The screenshot displays the 'Console Output' page for the 'terraform' job, build #13. The left sidebar shows navigation options: Status, Changes, Console Output (selected), Edit Build Information, Delete build '#13', Timings, Git Build Data, Pipeline Overview, Restart from Stage, Replay, Pipeline Steps, Workspaces, and Previous Build. The main area shows the console output text, which details the pipeline's execution steps, including checkout, git configuration, and fetching changes from a remote repository. The output is displayed in a light blue background with a green checkmark icon and a 'Download' button in the top right corner.

```

Terraform used the selected providers to generate the following execution
plan. Resource actions are indicated with the following symbols:
  [32m+ [0m create [0m

Terraform will perform the following actions:

[1m # aws_iam_user_policy_attachment.zoro_s3_full_access [0m will be created
[0m [32m+ [0m [0m resource "aws_iam_user_policy_attachment" "zoro_s3_full_access" {
  [32m+ [0m [0m id          = (known after apply)
  [32m+ [0m [0m policy_arn = "arn:aws:iam::aws:policy/AmazonS3FullAccess"
  [32m+ [0m [0m user       = "zoro"
}

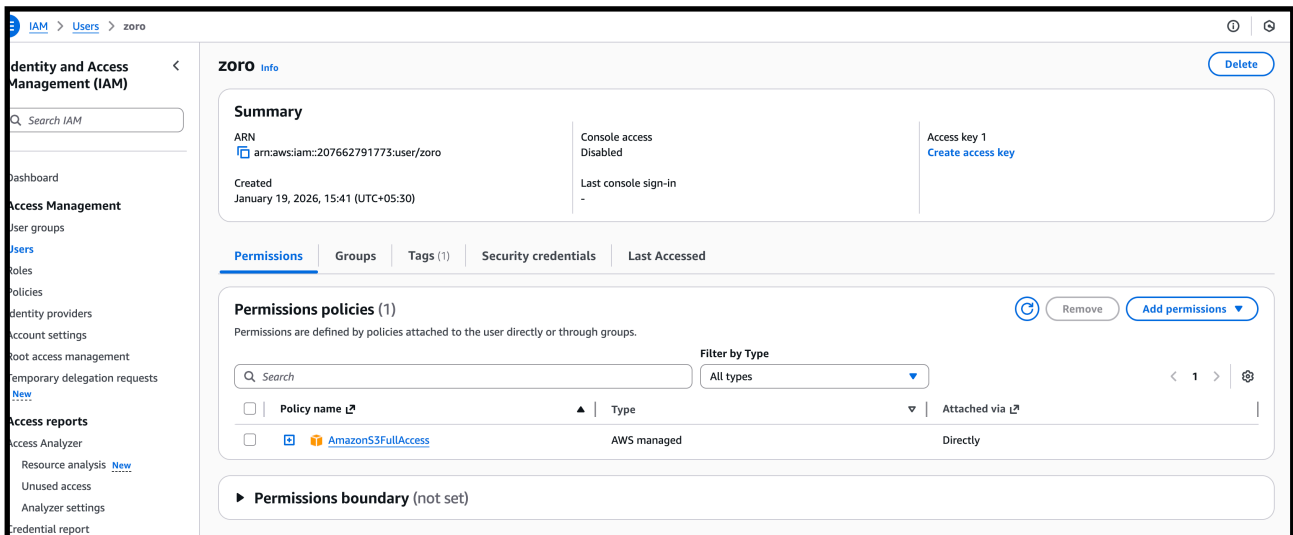
[1mPlan: [0m [0m1 to add, 0 to change, 0 to destroy.
[0m [1maws_iam_user_policy_attachment.zoro_s3_full_access: Creating... [0m [0m
[0m [1maws_iam_user_policy_attachment.zoro_s3_full_access: Creation complete after 0s [id=zoro-202601191015443206000000001] [0m
[0m [1m [32m
Apply complete! Resources: 1 added, 0 changed, 0 destroyed. [0m
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS

```

—-> pipeline success means we have successfully created a iam user named zoro in our aws account with least permission policy

—-> check your aws account —> iam —> users

ests	☐	waseem	/	0	 40 days ago	-	 40 da
	☐	zoro	/	0	-	-	-



—-> are able to see the policy which we have given to user is available here

4. Create a CI/CD pipeline for a Nodejs Application: <https://github.com/betawins/Trading-UI.git>

Set Up Jenkins

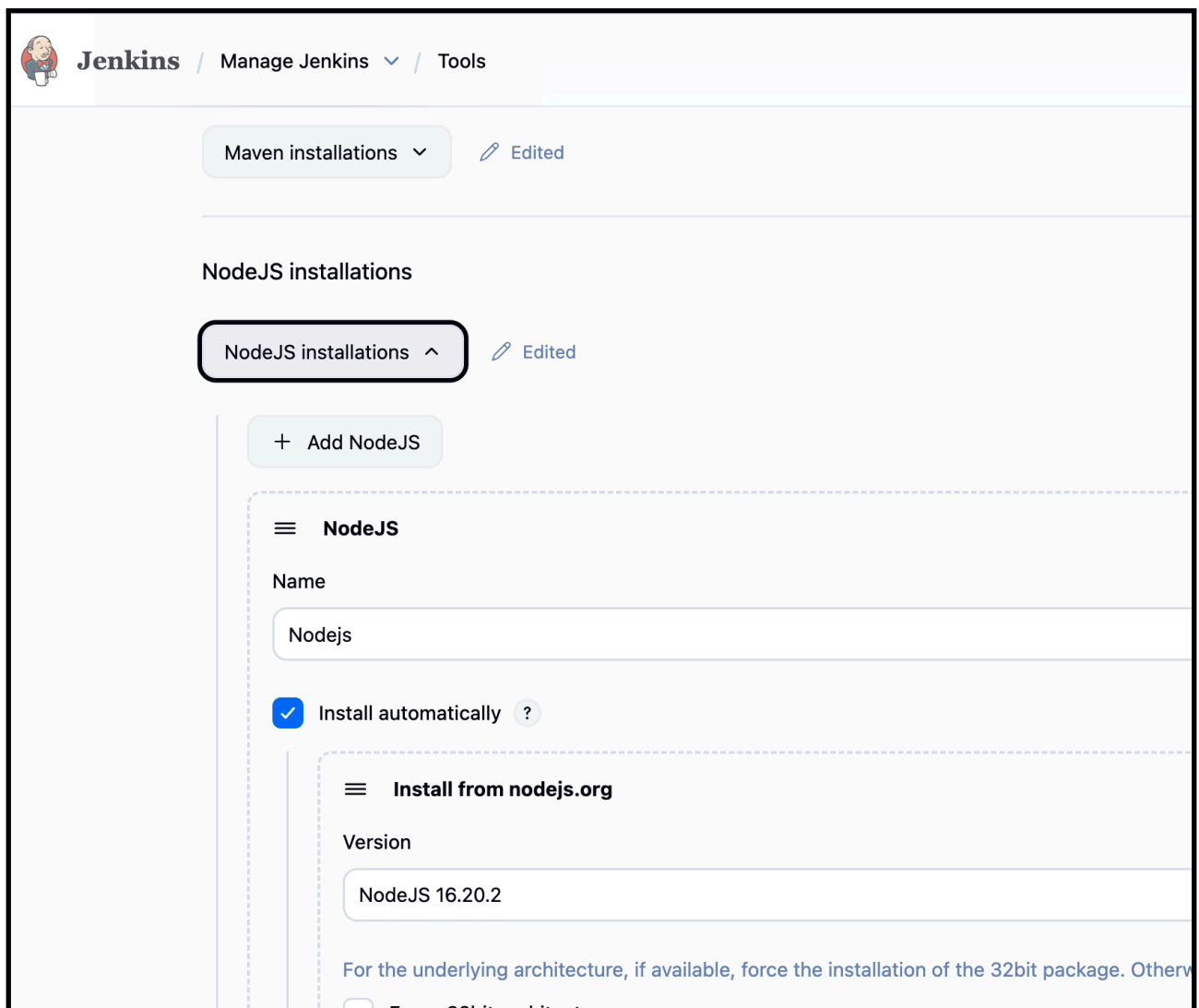
1. Install required plugins:

- **NodeJS plugin** (to install Node.js runtime)



Configure **NodeJS Tool** in Jenkins:

- Go to **Manage Jenkins → Global Tool Configuration → NodeJS installations**
- Add a version (e.g., NodeJS 16.x).



Fork the github repo of the url
. Create Jenkinsfile in GitHub Repo

The screenshot shows a GitHub repository named 'Trading-UI' by user 'Waseema761', which is a public fork of 'betawins/Trading-UI'. The repository has 1 branch (master) and is up to date. A commit titled 'Create Install nodejs and npn using linux.txt' by 'betawins' is highlighted, showing a diff with 13 additions and 0 deletions. The file list includes 'coverage', 'public', 'src', 'Install nodejs and npn using linux.txt', 'Jenkinsfile', 'README.md', 'package-lock.json', and 'package.json'. The README states the project was bootstrapped with 'Create React App'.

File	Commit Message	Time
coverage	final push	7 years ago
public	First UI code	7 years ago
src	config changes	7 years ago
Install nodejs and npn using linux.txt	Create Install nodejs and npn using linux.txt	5 years ago
Jenkinsfile	Update Jenkinsfile	6 years ago
README.md	First UI code	7 years ago
package-lock.json	Sprint 1 code	7 years ago
package.json	Sprint 1 code	7 years ago

—> Inside jenkinsfile

```
pipeline {
  agent any

  tools {
    nodejs "Nodejs"
  }

  stages {
    stage('Checkout') {
      steps {
        git url: 'https://github.com/Waseema761/Trading-UI.git',
branch: 'master'
      }
    }

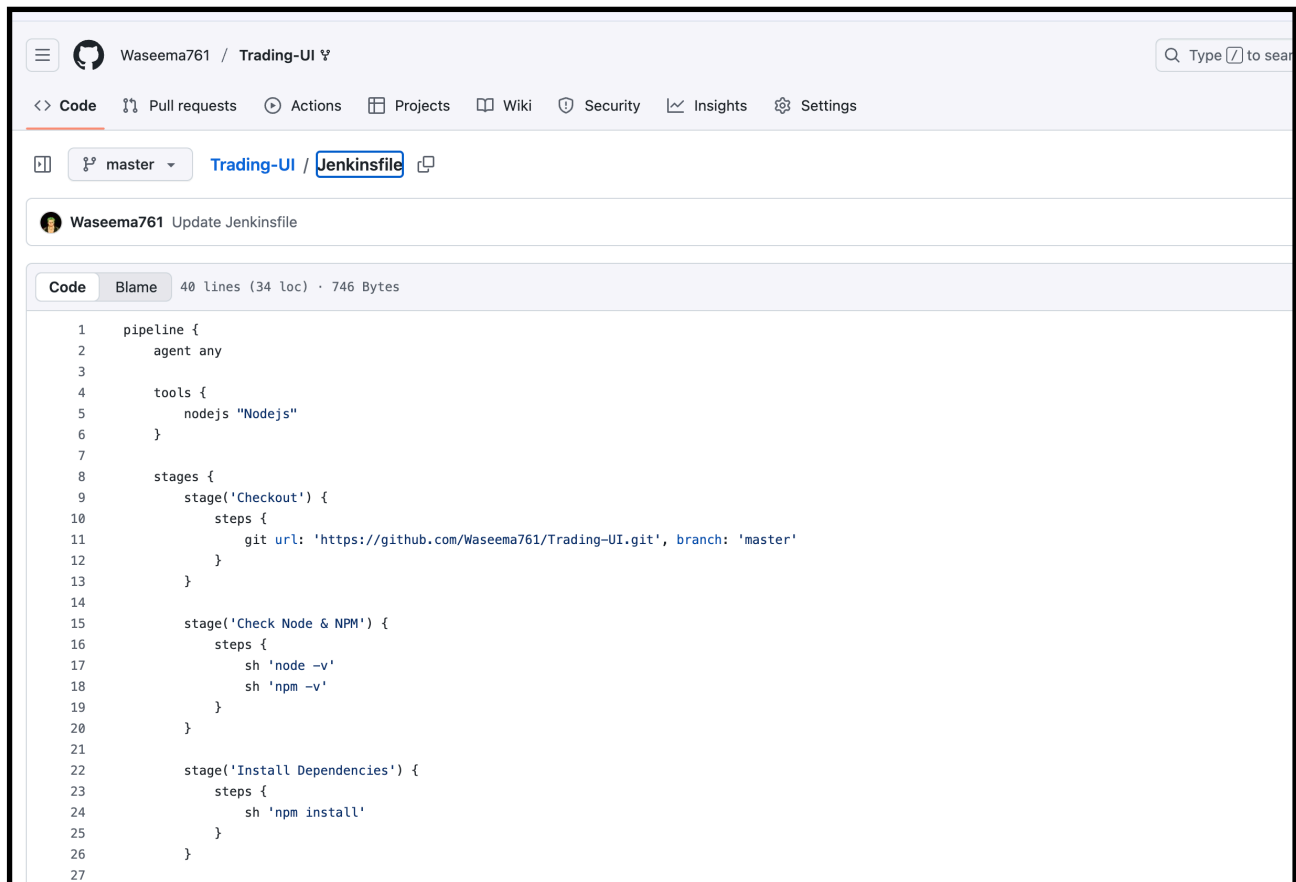
    stage('Check Node & NPM') {
      steps {
        sh 'node -v'
        sh 'npm -v'
      }
    }

    stage('Install Dependencies') {
      steps {
        sh 'npm install'
      }
    }

    stage('Build') {
      steps {
        sh 'CI=false npm run build'
      }
    }

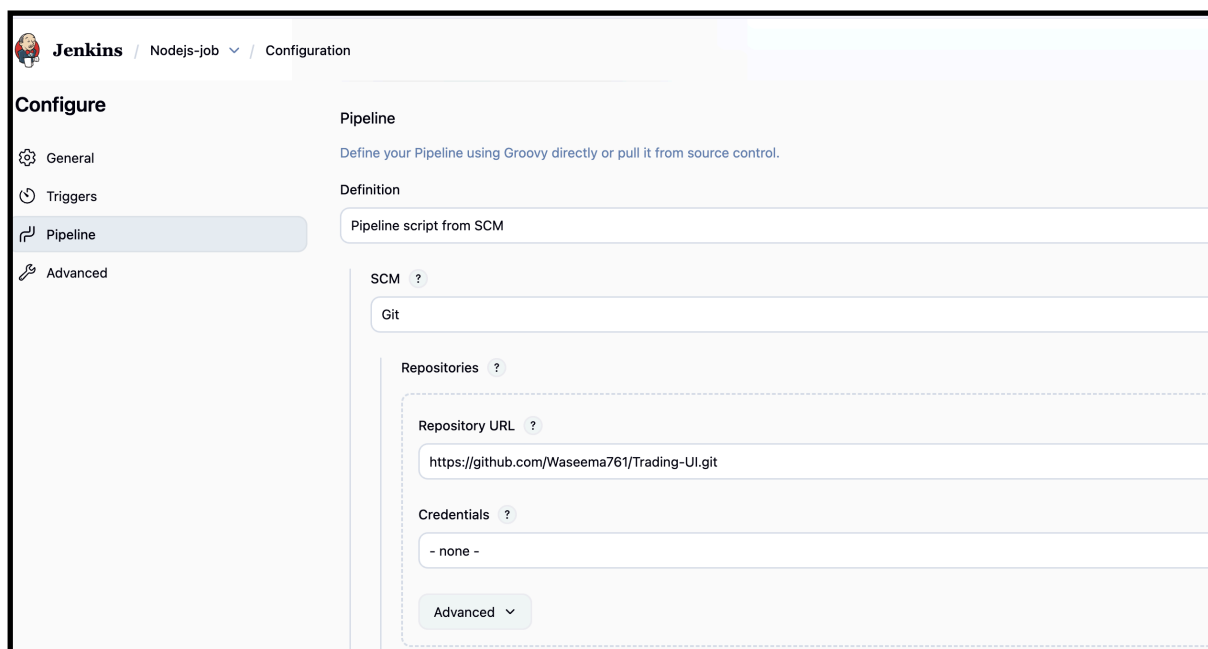
    stage('Test') {
      steps {
        sh 'npm test || echo "No tests found"'
      }
    }
  }
}
```

}



```
1 pipeline {
2   agent any
3
4   tools {
5     nodejs "Nodejs"
6   }
7
8   stages {
9     stage('Checkout') {
10      steps {
11        git url: 'https://github.com/Waseema761/Trading-UI.git', branch: 'master'
12      }
13    }
14
15    stage('Check Node & NPM') {
16      steps {
17        sh 'node -v'
18        sh 'npm -v'
19      }
20    }
21
22    stage('Install Dependencies') {
23      steps {
24        sh 'npm install'
25      }
26    }
27  }
```

- > now create a Job and give name Nodejs-job
- > attach the repo were we edited our jenkinsfile
 - New Item → Pipeline → Name it → Select "Pipeline from SCM"
 - Choose **Git** and provide repo URL.



Jenkins / Nodejs-job / Configuration

Configure

- General
- Triggers
- Pipeline**
- Advanced

Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script from SCM

SCM ?

Git

Repositories ?

Repository URL ?

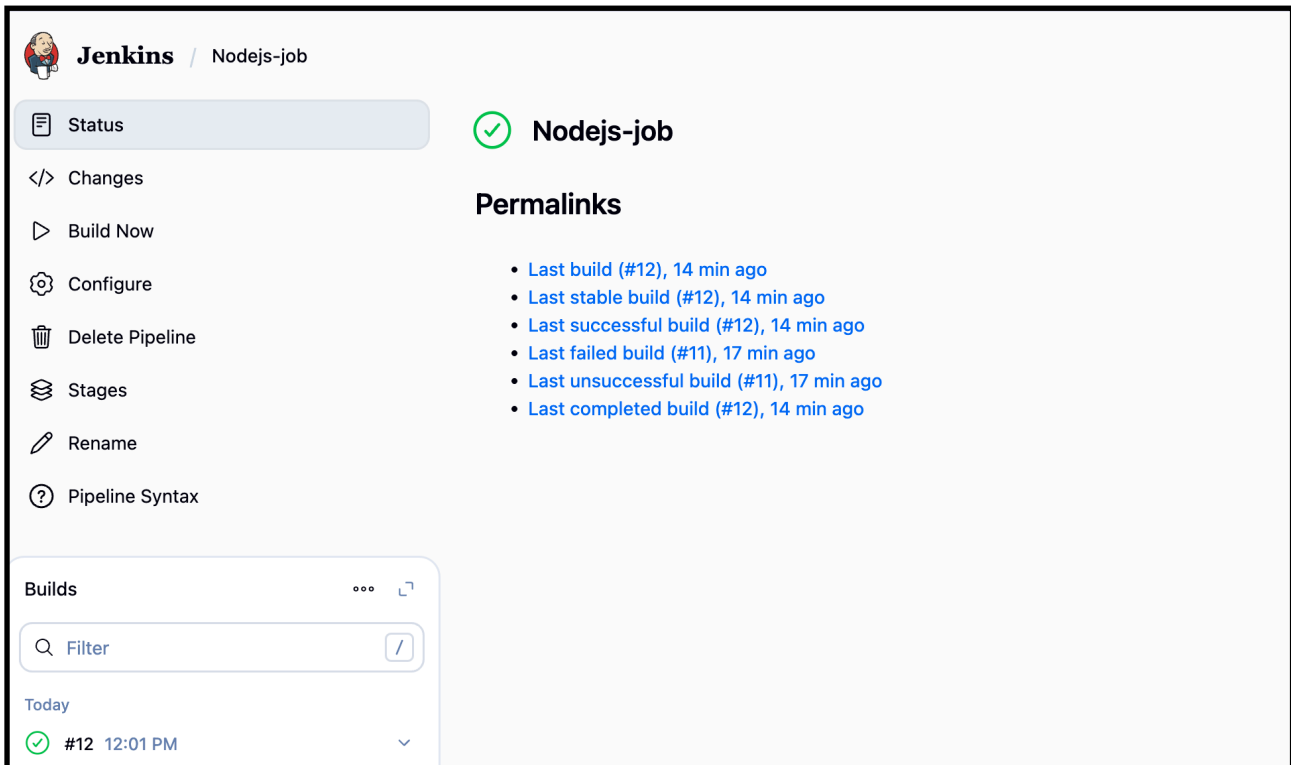
https://github.com/Waseema761/Trading-UI.git

Credentials ?

- none -

Advanced ▾

Save and build



The screenshot shows the Jenkins web interface for a job named 'Nodejs-job'. The left sidebar contains navigation links: Status (selected), Changes, Build Now, Configure, Delete Pipeline, Stages, Rename, and Pipeline Syntax. The main area displays the job's status as 'Nodejs-job' with a green checkmark icon. Below this, a 'Permalinks' section lists several links to previous builds, all indicating they were completed 14 or 17 minutes ago. At the bottom left, a 'Builds' section shows a list of builds, with the most recent build (#12) at 12:01 PM being the selected one.

Jenkins / Nodejs-job

Status

</> Changes

▶ Build Now

⚙️ Configure

🗑️ Delete Pipeline

📁 Stages

✎ Rename

❓ Pipeline Syntax

Builds

Filter

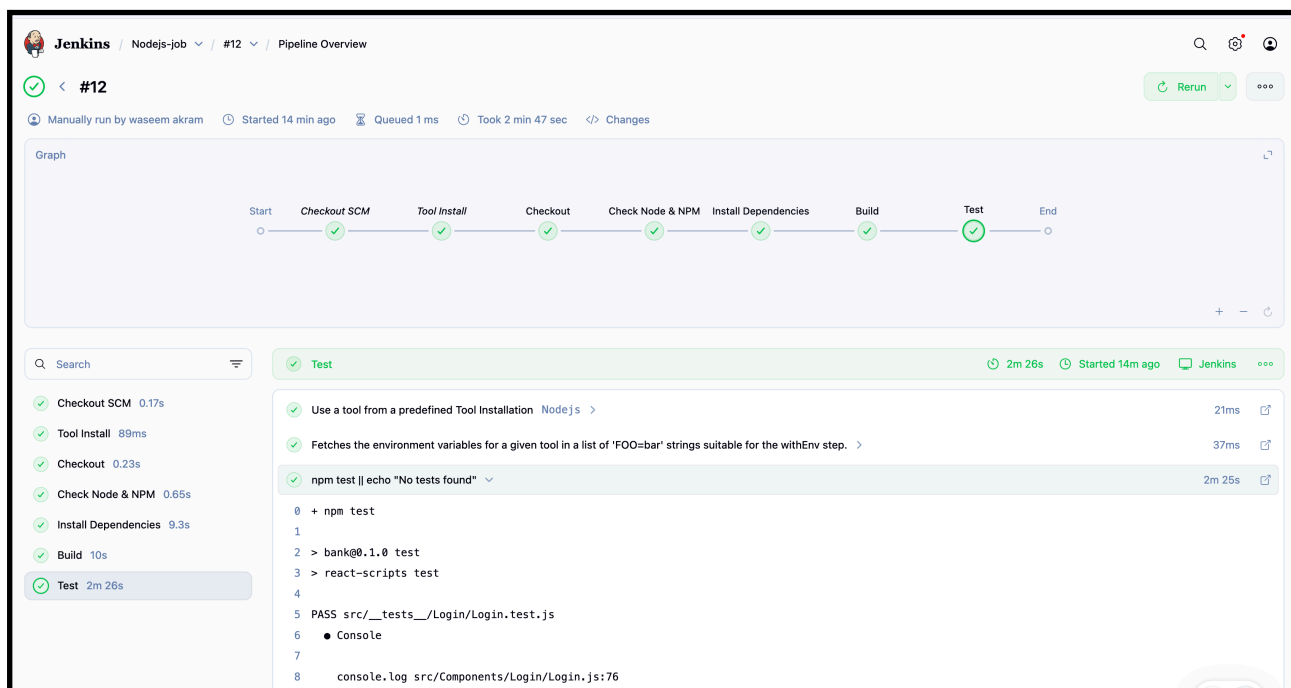
Today

✓ #12 12:01 PM

Nodejs-job

Permalinks

- [Last build \(#12\), 14 min ago](#)
- [Last stable build \(#12\), 14 min ago](#)
- [Last successful build \(#12\), 14 min ago](#)
- [Last failed build \(#11\), 17 min ago](#)
- [Last unsuccessful build \(#11\), 17 min ago](#)
- [Last completed build \(#12\), 14 min ago](#)



This screenshot provides a detailed view of the Jenkins pipeline for build #12. The top navigation bar shows the job name 'Nodejs-job' and the specific build '#12'. The pipeline graph at the top illustrates the sequence of steps: Start, Checkout SCM, Tool Install, Checkout, Check Node & NPM, Install Dependencies, Build, Test, and End. All steps are marked with green checkmarks, indicating a successful execution. Below the graph, a search bar and a list of steps are provided. The 'Test' step is currently selected, showing its duration of 2m 26s and the time it started 14 minutes ago. The console output for the 'Test' step is visible, showing the execution of 'npm test' and the successful completion of the test suite.

Jenkins / Nodejs-job / #12 / Pipeline Overview

✓ < #12

Manually run by waseem akram Started 14 min ago Queued 1 ms Took 2 min 47 sec </> Changes

Graph

Start Checkout SCM Tool Install Checkout Check Node & NPM Install Dependencies Build Test End

Search

✓ Checkout SCM 0.17s

✓ Tool Install 89ms

✓ Checkout 0.23s

✓ Check Node & NPM 0.65s

✓ Install Dependencies 9.3s

✓ Build 10s

✓ Test 2m 26s

✓ Test 2m 26s Started 14m ago Jenkins

Use a tool from a predefined Tool Installation Nodejs > 21ms

Fetches the environment variables for a given tool in a list of 'FOO=bar' strings suitable for the withEnv step. > 37ms

npm test || echo "No tests found" > 2m 25s

```
0 + npm test
1
2 > bank@0.1.0 test
3 > react-scripts test
4
5 PASS src/__tests__/Login/Login.test.js
6   ● Console
7
8   console.log src/Components/Login/Login.js:76
```

—> check cli of Jenkins workspace

```
first-job      hiring-app-parallel@tmp      parametrized-job2  private      scripted-pipeline1@tmp
[root@ip-172-31-65-24 workspace]# cd Nodejs-job
[root@ip-172-31-65-24 Nodejs-job]# ls
'Install nodejs and npn using linux.txt'  Jenkinsfile  README.md  build  coverage  node_modules  package-lock.json  pack
```

5.) Explain 10 Maven commands.

1. mvn clean

- Removes the **target/** directory (where compiled files, packaged JARs/WARs, etc. are stored).
- Ensures you start with a **clean build environment**.

mvn clean

2. mvn compile

- Compiles the **source code** of the project (from `src/main/java`).
- Output goes to `target/classes`.

mvn compile

3. mvn test

- Compiles and runs the **unit tests** (in `src/test/java`) using JUnit/TestNG.

mvn test

4. mvn package

- Packages the compiled code into a **JAR/WAR** file as defined in `pom.xml`.
- Example: `target/my-app-1.0-SNAPSHOT.jar`

mvn package

5. mvn install

- Installs the packaged project into the **local Maven repository** (`~/.m2/repository`).
- Makes the artifact available for use in other local projects.

mvn install

6. mvn deploy

- Copies the packaged code to a **remote repository** (like Nexus, Artifactory).
- Used in CI/CD pipelines for sharing artifacts with other developers.

mvn deploy

7. mvn site

- Generates a **site documentation** (reports, dependencies, plugins info, etc.) for the project in `target/site`.

mvn site

8. mvn dependency:tree

- Shows the **dependency hierarchy** of the project.
- Useful for identifying conflicts between library versions.

mvn dependency:tree

9. mvn validate

- Validates the project's structure and checks if pom.xml is correct.
- Runs before compilation.

mvn validate

10. mvn verify

- Runs any **integration tests** and checks if the package is valid.
- Ensures the project is ready for deployment.

mvn verify

Summary:

Command	Purpose
mvn clean	Remove target/ build files
mvn compile	Compile source code
mvn test	Run unit tests
mvn package	Package into JAR/WAR
mvn install	Install into local repo
mvn deploy	Deploy to remote repo
mvn site	Generate project site reports
mvn dependency:tree	Show dependency tree
mvn validate	Validate project structure
mvn verify	Run integration tests & verify package

